

PROGRAMACION MODULAR

Una estrategia para la resolución de problemas complejos con computadoras es la división o descomposición del problema en otros subproblemas mas pequeños. Estos subproblemas se implementan (realizan) mediante módulos o subprogramas. Normalmente un proyecto se compone de un programa principal y un conjunto de subprogramas. Los cuales son llamados desde el programa principal.

El proceso de descomposición de un problema en módulos se conoce como *modularización* y a la programación relativa a ellos *programación modular*. Los subprogramas se clasifican en procedimientos y funciones. Son similares, aunque presentan notables diferencias.

- Las funciones, normalmente, devuelven un solo valor. Los procedimientos pueden devolver 0, uno o varios valores.
- A un nombre de procedimiento no se le puede asignar un valor, y por consiguiente ningún tipo esta asociado con un nombre de procedimiento.
- Una función se referencia utilizando su nombre en una expresión, mientras que un procedimiento se referencia con una llamada o invocación al mismo.

Ventajas:

- El uso de procedimientos facilita el diseño descendente.
- Los procedimientos se pueden ejecutar mas de una vez en un programa y en diferentes programas, ahorrando en consecuencia tiempo de programación.
- El uso de procedimientos facilita la división de tareas en un grupo de programación
- Los procedimientos se pueden comprobar individualmente y estructurarse en librerías específicas.

Funciones propias de pascal

Funciones Matemáticas

Función	Descripción
Sin(Var);	Devuelve en radianes el seno de el ángulo Var
Cos(Var);	Devuelve en radianes el coseno de el ángulo Var
ArcTan(Var);	Devuelve en radianes el arco tangente de el ángulo Var
Exp(Var);	Devuelve el exponencial "e" de Var
Ln(Var);	Devuelve el logaritmo natural del argumento Var
Sqr(Var);	Devuelve el cuadrado de el argumento Var
Sqrt(Var);	Devuelve la raíz cuadrada del argumento Var
Abs(Var);	Devuelve el valor absoluto de Var
Round(Var);	Devuelve el redondeo de Var
Trunc(Var);	Devuelve a Var sin la fracción
Frac(Var);	Devuelve la parte decimal de un numero real
Int(Var);	Devuelve la parte entera de un real
Ord(Var);	Devuelve el valor asociado a un caracter ASCCI
Pi;	Devuelve el valor de Pi

Funciones de Cadena

Función	Descripcion
Clreol	Borra línea actual
Concat(Cad1,Cad2);	Une dos cadenas y el resultado lo deja en cadena1
Copy(Cad,indice,Lim);	Devuelve una cadena que va desde el índice hasta el lim
Delete(Cad,indice,Lim);	Borra una subcadena que se encuentra en cadena de índice hasta lim

Deline;	Borra línea de texto donde esta el cursor
Gotoxy(x,y);	Posiciona al cursor en x,y
Insert(Cad1,Cad2,indice);	Inserta en cad1 cad2 desde la posición de índice
Inslne	Inserta una línea vacía en la posición actual
Length(Cad);	Devuelve la longitud de la cadena
Pos(Cad1,Cad2);	Devuelve la posición de cad1 dentro de cad2
Read(Variable);	Lee una entrada y la coloca en una variable
Readln(Variable);	Lee una entrada y la coloca en una variable y retorna
Wherex;	Devuelve posición del cursor en la horizontal
Wherey;	Devuelve posición de el cursor en la vertical
Write(Variable);	Escribe el contenido de la variable en la pantalla
Writeln(Variable);	Escribe el contenido de la variable en la pantalla y retorna
Val(Cad,N,Num);	Convierte una cadena a un numero n si hay éxito Num = 0

Además de las funciones propias de pascal, es posible que el usuario pueda definir sus propias funciones.

Una función es un subprograma que devuelve un único resultado al programa o subprograma que lo llamó.

Declaración de una Función:

function nombre(<lista de parámetros formales>)
<declaraciones locales>

Begin

<cuerpo de la función>

Nombre:= valor de la función

End;

Ejemplo1

Function SacarDoble(x:integer):integer; Begin Doble:= x * 2; End;	SacarDoble(3); // Resultado 6
---	---

Ejemplo 2

Function Signo(x:integer): char; Begin if x = 0 then Signo:= '0'; if x > 0 then Signo:= '+'; if x < 0 then Signo:= '-'; End;	Signo(-345); // Resultado -
--	---------------------------------------

Declaración de un Procedimiento:

```
procedure nombre(<lista de parámetros formales>)  
<declaraciones locales>  
Begin  
  <cuerpo del procedimiento>  
End;
```

Invocación de procedimiento:

nombre (<lista de parámetros actuales>);

En resumen, un procedimiento consta de:

- Una cabecera del procedimiento y en caso de existir una lista de parámetros formales.
- Una sección de declaración local, que puede contener variables, constantes o inclusive otros procedimientos.
- Una sección ejecutable: <cuerpo del procedimiento>

Transferencia de información al / desde procedimiento: los parámetros.

Una de las características importantes de los subprogramas es la posibilidad de comunicación entre el programa principal y los subprogramas (procedimientos). Esta comunicación se realiza a través de una lista de parámetros.

Un **parámetro** es un método para pasar información (valores de variables) del programa principal a un procedimiento o viceversa.

Los procedimientos se clasifican en:

- Procedimientos sin parámetros (no existe comunicación entre el programa principal y los procedimientos o entre dos procedimientos).
- Procedimientos con parámetros (existe comunicación entre el programa principal y el procedimientos o entre procedimientos).

Ejemplo: Sin parámetros

<pre>procedure estrella(); Begin Writeln('*****'); End;</pre>	<pre>//Resultado *****</pre>
--	---------------------------------------

Ejemplo: Con parámetros

<pre>procedure estrella(cant:integer); var i:integer; Begin For i:=0 to cantidad -1 do Writeln('*****'); End;</pre>	<pre>Estrella(3); //Resultado ***** ***** *****</pre>
---	--

Parámetros actuales y formales

Los procedimientos constan de dos partes:

Parámetros actuales: son los valores que se pasan al procedimiento.

Parámetros formales: sirven para contener los valores de los parámetros actuales cuando se invoca el procedimiento.

Nota: los parámetros actuales deben coincidir en tipo, número y orden con los parámetros formales. Es decir debe haber una correspondencia entre parámetros.

Parámetros formales	Parámetros actuales
<pre>procedure mens (can:integer; m:string; n:integer); var i:integer; Begin For i:=0 to can -1 do begin Write(mens); num:=num +1; Writeln(num); End; End;</pre>	<pre>mens(3,'El numero es:',10); //Resultado El número es: 11 El número es: 12 El número es: 13</pre>

procedure mens (can:integer; m:string; n:integer);


mens(3 , 'El numero es:' , 10);

Parámetros por copia y por referencia

Por copia

Los parámetros por copia son parámetros unidireccionales que se utilizan para proporcionar información a un procedimiento. Estos no devuelven valores. Esto quiere decir que la variable dentro del procedimiento puede ser modificada sin que afecte a la variable que se pasó por parámetro desde el programa llamador.

Ejemplo (parámetros por copia)

<pre>procedure PorCopia (numero:integer); Begin Numero:= 2000; Writeln(numero); End;</pre>	<pre>Num:=10; PorCopia(num); Writeln(num); //Resultado 2000 10</pre>
---	---

Por referencia

Los parámetros por referencia se utilizan tanto para transmitir como para recibir valores entre el subprograma y el programa llamador. Puede actuar como parámetro de salida o de entrada/salida.

Para definir que el parámetro sea por referencia se agrega la cláusula **var** antes de la declaración de la variable.

Ejemplo (parámetros por referencia)

procedure PorReferencia (var numero:integer); Begin Numero:= 2000; Writeln(numero); End;	Num:=10; PorCopia(num); Writeln(num); //Resultado 2000 2000
---	--

Algunas reglas para saber cuando utilizar parámetros por copia o por referencia.

- Si la información que se pasa al procedimiento no tiene que ser devuelta fuera del procedimiento, puede ser un parámetro por valor. (entrada)
- Si se tiene que devolver información al programa llamador, el parámetro debe ser por referencia.(salida)
- Si la información que se pasa al procedimiento puede ser modificada, el parámetro debe ser pasado por referencia (entrada/salida).

Variables locales y globales

Las variables que intervienen en un procedimiento pueden ser de dos tipos: locales y globales.

Una variable local es una variable que esta declarada dentro de un subprograma y se dice que su ámbito es el del subprograma. Esta variable solo esta disponible durante el funcionamiento del mismo. Su valor se pierde luego de la terminación de ese subprograma.

Una variable global es una variable que esta declarada en el programa principal. Al contrario de las locales, estas variables pueden ser utilizadas dentro del programa principal como dentro de todos los subprogramas.

Regla: las variables globales se declaran en el programa principal, mientras que las variables locales se declaran en un subprograma.

Ejemplo:

Program variables; Var A:integer; B:integer; Procedure DemoA; var C:integer; D:integer; begin C:=10; D:=20; A:=30; writeln(A); writeln(B); end ; Begin A:=100; B:=200; D:=1000; B:=1000; DemoA writeln(A); writeln(B); End.	//resultado 30 200 30 200 //Observaciones: Como se puede ver el ámbito de las variables A y B es global. Mientras que el ámbito de las variables C y D son locales. Lo que significa que no se pueden acceder desde el programa principal. En el procedimiento DemoA se hace una asignación a las variable C,D locales y a las variable global A.
---	--